Modelo de datos por schemas

E-commerce en arquitectura de 5 nodos · Nodo L4 (PostgreSQL 18) · Base `Proyecto_Integrador`

6	10	10	0
schemas	tablas	llaves foráneas	tablas en public

1. Resumen

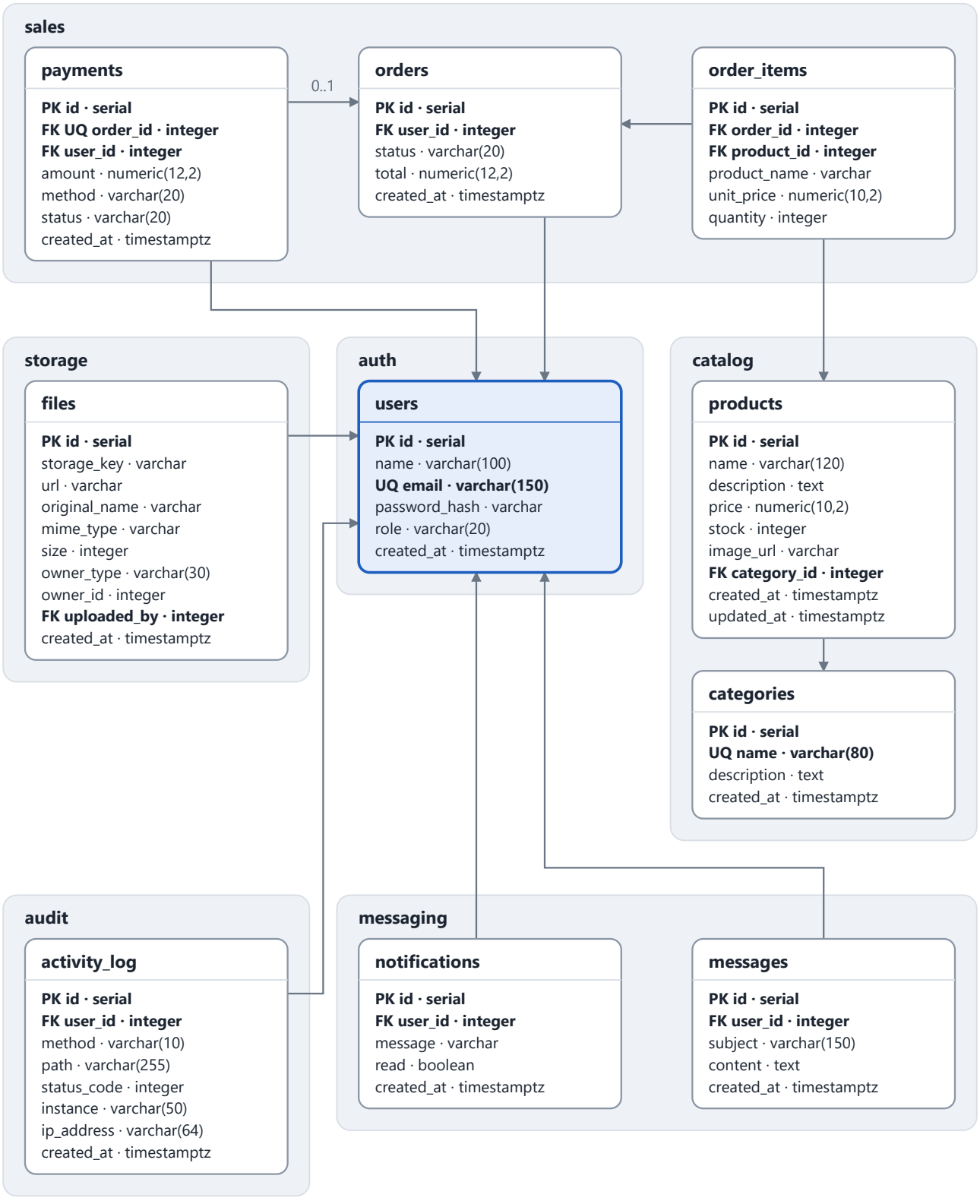
La base de datos se organiza en **6 schemas**, uno por área de responsabilidad del negocio, en lugar de dejar todas las tablas en `public`. Cada schema corresponde a los módulos del backend (L3) que lo gestionan, de modo que la capa de datos refleja la arquitectura lógica del sistema.

Schema	Responsabilidad	Tablas	Acceso
`auth`	Identidad y acceso	`users`	Privado
`catalog`	Catálogo de la tienda	`categories`, `products`	Lectura pública
`sales`	Operación comercial	`orders`, `order_items`, `payments`	Privado
`storage`	Archivos	`files`	Privado
`messaging`	Comunicación con el cliente	`notifications`, `messages`	Privado
`audit`	Auditoría	`activity_log`	Privado
`public`	Sin uso	ninguna	Sin acceso general

> Solo el backend se conecta a la base, con el usuario de aplicación `ecommerce_app`. No se usa el administrador `postgres`. El frontend nunca accede a la base: consume la API a través de NGINX.

2. Diagrama entidad–relación

Cada recuadro sombreado es un schema. `auth.users` es el centro del modelo: 6 de las 10 llaves foráneas apuntan a ella.



Cada flecha va de la llave foránea (lado N) a la clave primaria que referencia (lado 1).
PK clave primaria · FK llave foránea · UQ valor único · 0..1: un pedido tiene como máximo un pago.

3. Schemas y tablas

`auth` Identidad y acceso

Privado

Concentra los datos más sensibles del sistema: contraseñas cifradas y roles. Al estar separado se puede restringir quién lo lee y protegerlo con permisos propios, sin afectar al resto.

Módulos del backend que lo usan: **auth, users** · Tablas: **1**

`auth.users`

Clientes y administradores de la tienda.

Columna	Tipo	Clave	Qué guarda
`id`	`serial`	PK	Identificador del usuario
`name`	`varchar(100)`		Nombre completo
`email`	`varchar(150)`	UQ	Correo con el que inicia sesión. No se repite
`password_hash`	`varchar`		Contraseña cifrada con bcrypt. Nunca se guarda en texto plano
`role`	`varchar(20)`		Rol: 'customer' (cliente) o 'admin'. Por defecto 'customer'
`created_at`	`timestamptz`		Fecha de registro

`catalog` Catálogo de la tienda

Lectura pública

Es la información más consultada y la única que se muestra sin iniciar sesión. Cambia por gestión del administrador, no por las compras. Por eso es el único schema que admite un usuario de solo lectura.

Módulos del backend que lo usan: **products, catalog** · Tablas: **2**

`catalog.categories`

Categorías que agrupan los productos (Electrónica, Hogar, Ropa, Deportes...).

Columna	Tipo	Clave	Qué guarda
`id`	`serial`	PK	Identificador de la categoría
`name`	`varchar(80)`	UQ	Nombre de la categoría. No se repite
`description`	`text`		Descripción opcional
`created_at`	`timestamptz`		Fecha de creación

`catalog.products`

Productos en venta, con precio, stock e imagen.

Columna	Tipo	Clave	Qué guarda
`id`	`serial`	PK	Identificador del producto
`name`	`varchar(120)`		Nombre del producto
`description`	`text`		Descripción opcional
`price`	`numeric(10,2)`		Precio de venta
`stock`	`integer`		Unidades disponibles. Baja al crear un pedido
`image_url`	`varchar`		Ruta de la imagen guardada en el servidor de archivos (L5)
`category_id`	`integer`	FK	Categoría del producto (opcional) → catalog.categories
`created_at`	`timestamptz`		Fecha de creación
`updated_at`	`timestamptz`		Fecha de la última modificación

`sales` Operación comercial Privado

Agrupa las transacciones del negocio. Un pedido, su detalle y su pago se crean y validan juntos, por eso comparten schema. Es el historial de ventas: no debe perderse ni modificarse desde fuera.

Módulos del backend que lo usan: **orders, payments** · Tablas: **3**

`sales.orders`

Pedidos realizados por los usuarios.

Columna	Tipo	Clave	Qué guarda
`id`	`serial`	PK	Número de pedido
`user_id`	`integer`	FK	Usuario que hizo el pedido → auth.users
`status`	`varchar(20)`		Estado: 'pending', 'paid' o 'cancelled'. Por defecto 'pending'
`total`	`numeric(12,2)`		Total del pedido
`created_at`	`timestamptz`		Fecha del pedido

`sales.order_items`

Productos incluidos en cada pedido.

Columna	Tipo	Clave	Qué guarda
`id`	`serial`	PK	Identificador de la línea
`order_id`	`integer`	FK	Pedido al que pertenece → sales.orders
`product_id`	`integer`	FK	Producto comprado → catalog.products (queda vacío si el producto se elimina)
`product_name`	`varchar`		Copia del nombre al momento de la compra
`unit_price`	`numeric(10,2)`		Copia del precio al momento de la compra
`quantity`	`integer`		Cantidad comprada

`sales.payments`

Pago de cada pedido. Un pedido tiene como máximo un pago.

Columna	Tipo	Clave	Qué guarda
id	serial	PK	Identificador del pago
order_id	integer	FK UQ	Pedido pagado → sales.orders. Único: un pago por pedido
user_id	integer	FK	Usuario que pagó → auth.users
amount	numeric(12,2)		Monto pagado
method	varchar(20)		Método: 'card', 'cash' o 'transfer'
status	varchar(20)		Estado del pago. Por defecto 'approved'
created_at	timestamptz		Fecha del pago

`storage` Archivos Privado

El archivo físico vive en el servidor de archivos (L5); aquí solo queda su registro. Separarlo hace visible la frontera entre la base de datos y el almacenamiento externo.

Módulos del backend que lo usan: **files** · Tablas: **1**

`storage.files`

Registro de cada archivo almacenado en L5.

Columna	Tipo	Clave	Qué guarda
`id`	`serial`	PK	Identificador del registro
`storage_key`	`varchar`		Clave que devuelve L5 al guardar el archivo
`url`	`varchar`		Ruta para ver el archivo
`original_name`	`varchar`		Nombre original del archivo
`mime_type`	`varchar`		Tipo de archivo (image/png, image/jpeg...)
`size`	`integer`		Tamaño en bytes
`owner_type`	`varchar(30)`		A qué pertenece el archivo, por ejemplo 'product'
`owner_id`	`integer`		Identificador de ese dueño (referencia genérica, sin llave foránea)
`uploaded_by`	`integer`	FK	Usuario que lo subió → auth.users
`created_at`	`timestamptz`		Fecha de subida

`messaging` Comunicación con el cliente Privado

Son datos de interacción, de alto volumen y bajo valor transaccional. Pueden limpiarse o archivarse sin afectar las ventas ni el catálogo.

Módulos del backend que lo usan: **notifications, messages** · Tablas: **2**

`messaging.notifications`

Avisos que el sistema genera para el usuario.

Columna	Tipo	Clave	Qué guarda
`id`	`serial`	PK	Identificador del aviso
`user_id`	`integer`	FK	Usuario que lo recibe → auth.users
`message`	`varchar`		Texto del aviso (pedido registrado, pago aprobado...)
`read`	`boolean`		Si el usuario ya lo leyó. Por defecto false
`created_at`	`timestamptz`		Fecha del aviso

`messaging.messages`

Mensajes que los clientes envían a la tienda.

Columna	Tipo	Clave	Qué guarda
`id`	`serial`	PK	Identificador del mensaje
`user_id`	`integer`	FK	Usuario que lo envió → auth.users
`subject`	`varchar(150)`		Asunto
`content`	`text`		Contenido del mensaje
`created_at`	`timestamptz`		Fecha de envío

`audit` Auditoría Privado

Guarda el rastro de quién modificó qué. Separado del resto para que solo se pueda insertar y consultar, y para poder revisar la actividad sin tocar los datos del negocio.

Módulos del backend que lo usan: **audit (interceptor global)** · Tablas: **1**

`audit.activity_log`

Una fila por cada operación que modifica datos (crear, editar, borrar).

Columna	Tipo	Clave	Qué guarda
`id`	`serial`	PK	Identificador del registro
`user_id`	`integer`	FK	Usuario que hizo la operación → auth.users (vacío en registro y login)
`method`	`varchar(10)`		Tipo de operación: POST, PATCH o DELETE
`path`	`varchar(255)`		Ruta de la API utilizada
`status_code`	`integer`		Resultado: 201 creado, 400 rechazado, 500 error...
`instance`	`varchar(50)`		Instancia del backend que atendió: backend-1 o backend-2
`ip_address`	`varchar(64)`		Dirección de origen
`created_at`	`timestamptz`		Fecha y hora (con índice para consultar por fecha)

4. Claves y relaciones

Todas las tablas tienen una clave primaria `id` autoincremental. Hay 10 llaves foráneas; 7 de ellas cruzan de un schema a otro. Las reglas de borrado siguen el valor del dato: el historial de ventas se protege, y lo que solo tiene sentido junto a su padre se borra con él.

Tabla (FK)	Columna	Referencia (PK)	Cardinalidad	Al borrar el p
`catalog.products`	`category_id`	`catalog.categories.id`	1 categoría : N productos	**SET NULL** – E producto que sin categoría
`sales.orders`	`user_id`	`auth.users.id`	1 usuario : N pedidos	**RESTRICT** – N puede borrar usuario con pedidos
`sales.order_items`	`order_id`	`sales.orders.id`	1 pedido : N ítems	**CASCADE** – E detalle se bor con su pedid
`sales.order_items`	`product_id`	`catalog.products.id`	1 producto : N ítems	**SET NULL** – E ítem conserva nombre y pre
`sales.payments`	`order_id`	`sales.orders.id`	1 pedido : 0..1 pago	**RESTRICT** – U pedido pagad se puede bor
`sales.payments`	`user_id`	`auth.users.id`	1 usuario : N pagos	**RESTRICT** – N puede borrar usuario con p
`storage.files`	`uploaded_by`	`auth.users.id`	1 usuario : N archivos	**RESTRICT** – N puede borrar usuario con archivos
`messaging.notifications`	`user_id`	`auth.users.id`	1 usuario : N notificaciones	**CASCADE** – S borran con el usuario
`messaging.messages`	`user_id`	`auth.users.id`	1 usuario : N mensajes	**CASCADE** – S borran con el usuario
`audit.activity_log`	`user_id`	`auth.users.id`	1 usuario : N registros	**SET NULL** – E registro se conserva sin usuario

Restricciones únicas: `auth.users.email`, `catalog.categories.name` y `sales.payments.order_id`.

---

5. Schemas privados y de lectura pública

Usuario de PostgreSQL	Quién lo usa	Qué puede hacer
`ecommerce_app`	El backend (L3)	Dueño de los 6 schemas: lee y escribe en todos
`ecommerce_consulta`	Reportes o consultas externas	Solo **leer** el schema `catalog`. No entra a los demás ni puede modificar nada
`postgres`	Administrador de L4	Administración. No lo usa la aplicación

Los schemas `auth`, `sales`, `storage`, `messaging` y `audit` son **privados**: ningún otro usuario puede entrar. `catalog` es de **lectura pública** porque es la única información que la tienda muestra sin iniciar sesión. El usuario de consulta se crea con el script `database/permisos.sql`, que ejecuta el administrador.

Demostración, conectado como `ecommerce_consulta`:

```
SELECT * FROM catalog.products;                       -- funciona
SELECT * FROM auth.users;                             -- ERROR: permiso denegado al schema auth
INSERT INTO catalog.categories (name) VALUES ('x');   -- ERROR: permiso denegado
```

---

6. Consultas para comprobar la estructura

Sirven en el Query Tool de pgAdmin, en DBeaver o en `psql`, por si el árbol de pgAdmin no carga.

Schemas, su dueño y su descripción

```
SELECT n.nspname AS schema,
       pg_get_userbyid(n.nspowner) AS dueno,
       obj_description(n.oid, 'pg_namespace') AS descripcion
FROM pg_namespace n
WHERE n.nspname NOT LIKE 'pg_%' AND n.nspname <> 'information_schema'
ORDER BY 1;
```

Tablas de cada schema

```
SELECT table_schema AS schema, table_name AS tabla
FROM information_schema.tables
WHERE table_schema IN ('auth','catalog','sales','storage','messaging','audit','public')
ORDER BY 1, 2;
```

Columnas de una tabla

```
SELECT column_name AS columna, data_type AS tipo, is_nullable AS admite_nulo, column_default AS
valor_defecto
FROM information_schema.columns
WHERE table_schema = 'sales' AND table_name = 'orders'
ORDER BY ordinal_position;
```

Llaves foráneas y regla de borrado

```
SELECT tc.table_schema || '.' || tc.table_name   AS tabla,
       kcu.column_name                           AS columna,
       ccu.table_schema || '.' || ccu.table_name AS referencia,
       rc.delete_rule                            AS al_borrar
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu ON tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu ON tc.constraint_name = ccu.constraint_name
JOIN information_schema.referential_constraints rc ON rc.constraint_name = tc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY'
ORDER BY 1, 2;
```

Atajos en psql

```
\dn+                 -- lista de schemas con permisos y descripción
\dt auth.*           -- tablas del schema auth (cambiar por catalog, sales...)
\d sales.orders      -- columnas, claves y relaciones de una tabla
```

Datos de ejemplo cruzando schemas

```
-- Pedidos con el cliente (auth), el producto (catalog) y el pago (sales)
SELECT o.id AS pedido, u.name AS cliente, i.product_name AS producto,
       i.quantity AS cantidad, o.total, o.status, p.method AS pago
FROM sales.orders o
JOIN auth.users u            ON u.id = o.user_id
JOIN sales.order_items i     ON i.order_id = o.id
LEFT JOIN sales.payments p   ON p.order_id = o.id
ORDER BY o.id;

-- Últimas operaciones registradas y qué instancia las atendió
SELECT a.created_at, u.email, a.method, a.path, a.status_code, a.instance
FROM audit.activity_log a
```

```
LEFT JOIN auth.users u ON u.id = a.user_id
ORDER BY a.id DESC
LIMIT 20;
```

---

7. Cómo se crea esta estructura

De dos formas equivalentes:

- **Desde el backend:** al arrancar, la instancia `backend-1` crea los schemas que falten (`CREATE SCHEMA IF NOT EXISTS`), sincroniza las tablas a partir de las entidades de TypeORM y escribe las descripciones. La instancia `backend-2` no modifica la estructura, para que las dos no choquen.
- **Con el script** `database/schema.sql`, que crea lo mismo a mano sobre una base vacía.
